МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи №6

«Автоматизоване тестування Python-програм»

з дисципліни «Професійний Python»

Варіант №5 — «Каталог автомобілів»

Виконав: студент групи ФЕП-32

Горин Михайло

Репозиторій: github.com/MykhailoG212/Prof_python

Львів — 2026

1. Тема, мета та номер варіанта

Тема: автоматизоване тестування Python-програм із pytest. Мета — спроєктувати ізольовані unit та integration tests, перевірити звичайні, граничні й помилкові сценарії, файлові операції, конфігурацію та взаємодію компонентів, а також оцінити statement і branch coverage.

Індивідуальний варіант №5 — «Каталог автомобілів», наскрізний навчальний Python-проєкт попередніх лабораторних робіт.

2. Короткий опис проєкту

Car Catalog зберігає й аналізує каталог автомобілів. У роботі перевірено предметні моделі, application service та repository, потокову обробку CSV, валідацію, YAML-конфігурацію, tolerant/strict імпорт, журналювання й атомарний JSON-експорт. Зовнішня мережа не потрібна: файлові integration tests використовують тимчасові файли, а залежність repository підмінюється Mock.

3. Структура production code і тестів

```
src/car_catalog/
├── models.py, services.py, repositories.py
├── application.py, criteria.py, protocols.py
├── reliable/
│   ├── config.py, importers.py, exporters.py
│   ├── services.py, exceptions.py, main.py
└── streaming/
    ├── readers.py, parsers.py, validation.py
    ├── pipeline.py, filters.py, analytics.py

tests/
├── conftest.py
├── test_services.py, test_main.py, test_oop_model.py
├── test_processors_analytics.py, test_streaming.py
├── test_reliable_importer.py
├── test_reliable_validation.py, test_reliable_main.py
└── test_application_interactions.py
```

Тести згруповані за production-компонентами в окремі файли; integration-сценарії мають pytest marker integration. Shared fixtures розміщені у conftest.py та доступні за автоматичним пошуком pytest.

4. Матриця тестування

Категорія	Перевірка / приклад	Результат
Normal	Модель Car, пошук, статистика, tolerant import	Очікувані моделі, результати та JSON
Minimum/maximum boundary	1886 і 2026; ціна 0 і 1 000 000	Межі включені
Boundary outside	1885, 2027, ціна нижче/вище меж, NaN	RecordValidationError
Empty	Порожній каталог, порожні генератори	Нульові агрегати / None без помилок
Invalid input	Некоректний рік, CSV поля, YAML	Перевірений виняток та повідомлення
Dependency failure	Repository.add піднімає OSError	Помилка передається; ID доступний для retry
File/config	tmp_path: YAML, CSV, JSON; атомарне перейменування	Ізольовані файли й збереження попереднього output
Interaction	Mock/spec, return_value, side_effect	Перевірені аргументи, кількість і відсутність викликів
Integration	CSV → parser/validator → service → JSON	6 маркованих integration cases

		проходять
Repeated/isolated	Повторний service.add після помилки	Тести незалежні, ID не пропускається

5. Приклади реалізованих тестів

5.1 Unit test, fixture factory та fixture scope

```
from car_catalog.models import Car

@pytest.fixture
def car_factory() -> Callable[..., Car]:
    def create_car(*, brand="Toyota", model="Camry", year=2020,
                   price=25_000.0, mileage=45_000) -> Car:
        return Car(brand, model, year, price, mileage)
    return create_car

@pytest.fixture(scope="session")
def processing_config() -> ProcessingConfig:
    return ProcessingConfig(... )
```

car_factory має function scope за замовчуванням і створює новий об'єкт на запит. processing_config має session scope, оскільки це frozen dataclass без спільного змінюваного стану. Fixtures не імпортуються безпосередньо з conftest.py.

5.2 Parametrization, ids і boundary tests

```
@pytest.mark.parametrize(
    ("year", "price"),
    [(1886, "0"), (2026, "1000000")],
    ids=["minimum-boundary", "maximum-boundary"],
)
def test_parse_vehicle_accepts_inclusive_configuration_boundaries(...):
    vehicle = parse_vehicle(row, line_number=2, config=processing_config)
    assert vehicle == Vehicle(1, "Toyota", "Camry", year, float(price), 0)
```

Окремі параметризовані invalid cases перевіряють 1885/2027, від'ємну або завелику ціну, NaN і від'ємний пробіг.

5.3 pytest.raises, pytest.approx і tmp_path

```
with pytest.raises(RecordValidationError, match=message):
    parse_vehicle(row, line_number=2, config=processing_config)

assert calculate_average_price(cars) == pytest.approx(51666.67, 0.01)

config_path = tmp_path / "config.yaml"
input_path = tmp_path / "vehicles.csv"
write_csv(input_path)
config = load_config(config_path)
report = import_catalog(config)
assert json.loads(config.output_path.read_text(encoding="utf-8"))["statistics"] == {
    "valid": 2, "invalid": 1
}
```

pytest.raises перевіряє тип і текст domain exception. pytest.approx задає абсолютну похибку для середнього ціни. tmp_path створює ізольовані тимчасові файли; pytest керує їхнім життєвим циклом, тому окрема yield-fixture для cleanup тут не потрібна.

5.4 monkeypatch: environment variable та функція

```
monkeypatch.setenv("CAR_CATALOG_CONFIG", "custom/config.yaml")
assert reliable_main.main() == 0
assert loaded_paths == [Path("custom/config.yaml")]

monkeypatch.delenv("CAR_CATALOG_CONFIG", raising=False)
# CLI повертається до config/lab5.yaml

monkeypatch.setattr("builtins.input", lambda _prompt: next(answers))
monkeypatch.setattr("car_catalog.main.run_lab3_demo", run_demo)
```

CAR_CATALOG_CONFIG є конфігураційним override CLI; без змінної використовується config/lab5.yaml. monkeypatch автоматично відновлює середовище й підмінені об’єкти після тесту.

5.5 Mock, MagicMock, return_value, side_effect та interactions

```
repository = MagicMock(spec=VehicleRepository)
repository.all.return_value = [matching, another_car]
service = VehicleCatalogService(repository)
assert service.search_by_brand_or_model(brand="tesla") == [matching]
repository.all.assert_called_once_with()
repository.add.assert_not_called()

repository = Mock(spec=VehicleRepository)
repository.add.side_effect = OSError("storage unavailable")
with pytest.raises(OSError, match="storage unavailable"):
    service.add(vehicle)
repository.add.assert_called_once_with(1, vehicle)
```

Mock/spec обмежує інтерфейс залежності; return_value моделює читання repository, side_effect — збій на системній межі. Тест перевіряє результат, виклик та відсутність запису там, де його не очікують. Failure injection також підтверджує повторну спробу з тим самим ID.

5.6 Integration tests

```
@pytest.mark.integration
def test_tolerant_import_exports_only_valid_records_and_logs_error(tmp_path):
    config_path = tmp_path / "config.yaml"
    input_path = write_config(config_path)
    write_csv(input_path)
    config = load_config(config_path)
    report = import_catalog(config)
    assert (report.valid_count, report.invalid_count) == (2, 1)
    payload = json.loads(config.output_path.read_text(encoding="utf-8"))
    assert [row["vehicle_id"] for row in payload["vehicles"]] == [1, 2]
```

Це перевіряє кілька компонентів з реальною файловою системою, але тільки всередині tmp_path. Окремий strict сценарій перевіряє зупинку, chaining та збереження старого експорту. Мережеві API в проєкті відсутні, тому тести не залежать від Internet.

5.7 AsyncMock: аргументоване пояснення відсутності

Production-код каталогу синхронний: repository, CSV/YAML/JSON та CLI не мають async API чи coroutine dependency. Додавання штучного AsyncMock-тесту не перевіряло б реальну поведінку застосунку й суперечило б вимозі не over-mock-ати. Якщо з’явиться async API, потрібно перевіряти результат await і assert_awaited_once_with, а також failure через side_effect; для поточного варіанта обґрунтовано залишено синхронні mocks.

6. Запуск тестів і результати

Одна команда повного запуску: python -m pytest. Фактичний результат запуску під час формування звіту: 78 passed, 0 failed; integration: 6 passed; unit та решта тестів: 72 passed. Середовище: Python 3.14.3, pytest із pytest-cov. Поріг повного suite у pyproject.toml — 80%.

Окремі команди: python -m pytest tests/test_services.py; python -m pytest tests/test_services.py::test_calculate_average_price; python -m pytest -m integration --cov-fail-under=0. Для subset-запуску поріг вимкнено, бо частковий набір не має представляти coverage всього пакета.



Рисунок 1 — Фактичний результат повного запуску pytest і згенерований coverage.

Муру: Success: no issues found in 7 source files.

7. Coverage: statement, branch та HTML

Звіт покриття охоплює 1136 statements і 282 branch outcomes у пакеті car_catalog. Виконано statements: 87.94%; branch outcomes: 225/282 = 79.79%; combined coverage: 86.32%. У повному тестовому наборі пропущено 137 statement lines та 57 branch outcomes.

Модуль	Statements	Branches	Combined
src\car_catalog__init__.py	100%	—	100%
src\car_catalog\analytics.py	100%	10/10	100%
src\car_catalog\application.py	100%	—	100%
src\car_catalog\benchmark.py	73%	8/12	71%
src\car_catalog\criteria.py	100%	—	100%
src\car_catalog\decorators.py	100%	4/4	100%
src\car_catalog\dto.py	100%	—	100%
src\car_catalog\main.py	92%	36/44	89%
src\car_catalog\models.py	100%	14/14	100%
src\car_catalog\oop_demo.py	29%	1/2	32%
src\car_catalog\processors.py	100%	2/2	100%
src\car_catalog\protocols.py	100%	—	100%
src\car_catalog\reliable__init__.py	100%	—	100%
src\car_catalog\reliable\benchmark.py	80%	12/18	78%
src\car_catalog\reliable\config.py	77%	21/34	73%
src\car_catalog\reliable\exceptions.py	100%	—	100%
src\car_catalog\reliable\exporters.py	83%	3/6	79%
src\car_catalog\reliable\importers.py	88%	22/28	86%
src\car_catalog\reliable\main.py	94%	1/2	89%
src\car_catalog\reliable\models.py	100%	—	100%
src\car_catalog\reliable\services.py	93%	9/10	93%
src\car_catalog\repositories.py	95%	4/4	96%
src\car_catalog\services.py	100%	2/2	100%
src\car_catalog\streaming__init__.py	100%	—	100%
src\car_catalog\streaming__main__.py	0%	—	0%
src\car_catalog\streaming\analytics.py	100%	10/10	100%
src\car_catalog\streaming\batches.py	100%	6/6	100%
src\car_catalog\streaming\benchmark.py	82%	12/18	79%

src\car_catalog\streaming\demo.py	39%	1/6	35%
src\car_catalog\streaming\filters.py	100%	8/8	100%
src\car_catalog\streaming\iterators.py	94%	12/14	91%
src\car_catalog\streaming\models.py	100%	—	100%
src\car_catalog\streaming\parsers.py	100%	—	100%
src\car_catalog\streaming\pipeline.py	100%	—	100%
src\car_catalog\streaming\readers.py	100%	3/4	93%
src\car_catalog\streaming\transformations.py	100%	2/2	100%
src\car_catalog\streaming\validation.py	89%	22/22	92%

HTML-звіт створено у htmlcov/index.html; він показує деталі за файлами та пропущені рядки. Нижче наведено фрагмент фактичного HTML-звіту, відрендерений на основі його таблиці.

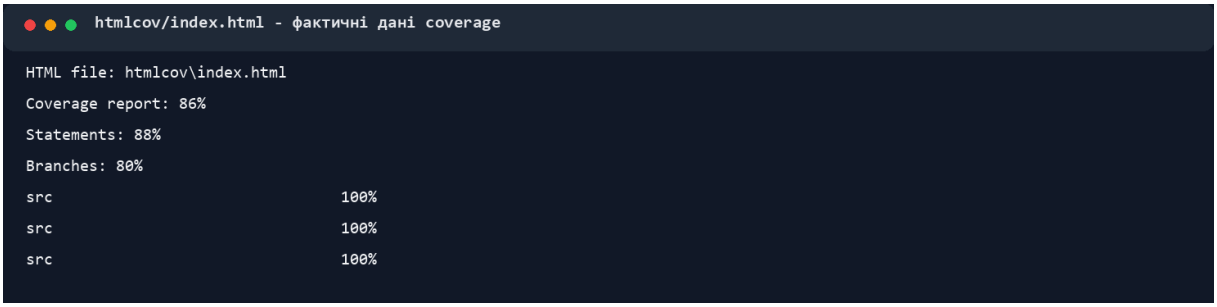


Рисунок 2 — Фрагмент HTML coverage report, сформованого pytest-cov.

```
<h1>Coverage report:
            <span class="pc_cov">86%</span>
        </h1>


Повний звіт: htmlcov/index.html
Команда: python -m pytest --cov=car_catalog --cov-branch --cov-report=term-missing --cov-report=html
```

8. Аналіз недостатньо протестованих paths

Найменше branch coverage мають інтерактивний legacy CLI та демонстраційні/benchmark paths. Вони виконують UI-вивід, запуск великих benchmark-наборів і окремі сценарії меню; ці гілки не визначають критичний контракт CSV/YAML/JSON. Конфігураційні та export edge paths також залишаються у списку term-missing.

• src\car_catalog\streaming\demo.py: branch coverage 16.7%, 5 невиконаних outcomes

• src\car_catalog\oop_demo.py: branch coverage 50.0%, 1 невиконаних outcomes

• src\car_catalog\reliable\exporters.py: branch coverage 50.0%, 3 невиконаних outcomes

• src\car_catalog\reliable\main.py: branch coverage 50.0%, 1 невиконаних outcomes

• src\car_catalog\reliable\config.py: branch coverage 61.8%, 13 невиконаних outcomes

• src\car_catalog\benchmark.py: branch coverage 66.7%, 4 невиконаних outcomes

• src\car_catalog\reliable\benchmark.py: branch coverage 66.7%, 6 невиконаних outcomes

• src\car_catalog\streaming\benchmark.py: branch coverage 66.7%, 6 невиконаних outcomes

• src\car_catalog\streaming\readers.py: branch coverage 75.0%, 1 невиконаних outcomes

• src\car_catalog\reliable\importers.py: branch coverage 78.6%, 6 невиконаних outcomes

Наступне доцільне посилення — параметризувати решту меню invalid inputs, конфігураційні помилки кожного типу й помилки файлової системи. Умова лабораторної — проаналізувати пропуски та запропонувати тести; не створювати формальні assertions заради 100%. Налаштований поріг 80% запобігає регресії загального coverage.

9. Перевірка реальних defects

Під час цього запуску тести не відтворили нових production defects; тому звіт не приписує собі непідтверджені виправлення. Перевірені ризикові сценарії включають пошкоджений YAML/CSV, строгий та tolerant імпорт, атомарне збереження попереднього JSON після exception, точність агрегатів та помилку repository. Доданий failure-injection test зафіксував регресійний контракт: якщо repository.add завершується помилкою, наступна спроба використовує той самий ID. Якщо вимога трактує «знайдені defects» як обов'язкове відтворення реального дефекту, у межах виміряних запусків такого дефекту не підтверджено.

10. Відповіді на контрольні запитання

1. Що таке automated test? Це код, який автоматично запускає перевірку очікуваної поведінки програми та сигналізує про невідповідність.

2. Які основні переваги автоматизованого тестування? Воно швидко виявляє defects, захищає від regressions, документує поведінку та спрощує безпечний refactoring.

3. Що таке regression? Це раніше справна поведінка, яку порушила нова зміна коду.

4. Що таке unit test? Перевірка однієї невеликої одиниці поведінки ізольовано від сторонніх компонентів.

5. Що таке integration test? Перевірка взаємодії кількох компонентів, наприклад CSV-читача, валідатора та JSON-експортера.

6. Чим integration test відрізняється від unit test? Integration test охоплює кілька компонентів і їхні межі; unit test перевіряє один компонент, зазвичай із підміною залежностей.

7. Що показує test pyramid? Рекомендоване співвідношення: багато швидких unit-тестів, менше integration-тестів і ще менше дорогих end-to-end-тестів.

8. Чому unit tests зазвичай більше? Вони швидші, дешевші, детермінованіші та точніше вказують на місце дефекту.

9. Що таке pytest? Це Python-фреймворк для автоматизованого тестування з assert-перевірками, fixtures, параметризацією та плагінами.

10. Як pytest знаходить tests? За типовими шаблонами: файли test_*.py або *_test.py та функції/методи з префіксом test_.

11. Якою командою запускається pytest? У цьому проєкті: python -m pytest; встановлені addopts також формують coverage-звіти.

12. Для чого pytest -v? Для детального виводу з окремою назвою та статусом кожного тесту.

13. Для чого pytest -x? Щоб зупинити запуск після першого невдалого тесту.

14. Що означає Arrange–Act–Assert? Arrange готує дані, Act виконує дію, Assert перевіряє результат.

15. Для чого використовується assert? Він фіксує очікувану умову; pytest показує відмінності, коли умова хибна.

16. Що таке assertion failure? Це невиконана перевірка, яка доводить розбіжність між фактичним і очікуваним результатом.

17. Чому floats не завжди слід порівнювати через ==? Двійкове представлення десяткових дробів може давати малу похибку округлення.

18. Для чого pytest.approx? Для порівняння чисел або числових послідовностей із допустимою похибкою.

19. Для чого pytest.raises? Для перевірки, що конкретна дія піднімає очікуваний тип винятку.

20. Як перевірити exception message? Задати match у pytest.raises або перевірити текст отриманого винятку через exc_info.

21. Як отримати exception object через exc_info? Використати `with pytest.raises(Type) as exc_info:` і звернутися до `exc_info.value`.

22. Що таке boundary testing? Це перевірка значень на межах діапазону та безпосередньо поруч із ними.

23. Які boundaries потрібно тестувати для діапазону 0–100? Щонайменше -0.01, 0, значення всередині, 100 і 100.01; за потреби перевірити точність на дробових межах.

24. Що таке parametrization? Один тестовий шаблон запускається багато разів із різними наборами вхідних даних.

25. Для чого pytest.mark.parametrize? Для оголошення таких наборів параметрів і очікуваних результатів без дублювання тестового коду.

26. Що дають ids параметризованих tests? Людинозрозумілі назви кожного випадку у виводі pytest.

27. Що таке fixture? Функція pytest, що готує та повертає ізольовані дані або ресурс для тесту.

28. Для чого використовуються fixtures? Для повторного використання setup, зменшення дублювання та контрольованого життєвого циклу ресурсів.

29. Що означає fixture setup? Підготовка стану або ресурсу до виконання тесту.

30. Що означає fixture teardown? Звільнення ресурсу або прибирання стану після тесту.

31. Для чого yield у fixture? Щоб передати підготовлений ресурс тесту, а після його завершення виконати cleanup-код.

32. Які fixture scopes підтримує pytest? function, class, module, package та session.

33. Який scope є default? function — fixture створюється заново для кожного тесту.

34. Для чого conftest.py? Для спільних fixtures та pytest hooks, доступних тестам відповідної теки без ручного імпорту.

35. Чому fixtures із conftest.py не імпортують вручну? Pytest автоматично знаходить їх у conftest.py відповідно до розташування тесту.

36. Що таке tmp_path? Вбудована function-scoped fixture pytest, яка надає унікальну тимчасову директорію як pathlib.Path.

37. Чому tmp_path корисний? Він ізолює файлові тести, запобігає конфліктам і забрудненню робочих даних проєкту.

38. Що таке test isolation? Незалежність тесту від порядку запуску, результатів інших тестів і спільного змінюваного стану.

39. Для чого monkeypatch? Тимчасово замінювати environment variables, атрибути, функції, словники або робочу теку з автоматичним відновленням.

40. Як monkeypatch змінює environment variable? Викликом `monkeypatch.setenv("NAME", "value")`; зміна діє лише під час тесту.

41. Як видалити environment variable у test? Викликом `monkeypatch.delenv("NAME", raising=False)`.

42. Як підмінити function через monkeypatch? Викликом `monkeypatch.setattr(module, "function", fake_function)` у модулі, де це ім'я використовується.

43. Що таке test double? Узагальнена підміна реальної залежності в тесті.

44. Що таке Dummy? Простий об'єкт-заповнювач, переданий лише для заповнення аргументу і не задіяний.

45. Що таке Stub? Підміна, яка повертає заздалегідь задані відповіді.

46. Що таке Fake? Спрощена робоча реалізація залежності, наприклад in-memory repository.

47. Що таке Mock? Підміна, що зберігає інформацію про взаємодію та дає змогу перевірити виклики.

48. Що таке Spy? Обгортка над реальною поведінкою, яка водночас записує виклики.

49. Для чого Mock? Ізолювати зовнішню залежність і перевірити, чи вона викликана з правильними аргументами та кількістю разів.

50. Що таке spec? Обмеження mock доступними атрибутами інтерфейсу реального об'єкта.

51. Для чого return_value? Задати значення, яке mock повертає при виклику.

52. Для чого side_effect? Задати виняток, послідовність відповідей або функцію, що виконується при виклику mock.

53. Як перевірити, що method викликано один раз? Використати `mock.method.assert_called_once_with(...)` або `assert_called_once()`.

54. Як перевірити parameters виклику? Порівняти виклик через `assert_called_once_with(expected_args, expected_kwargs)`.

55. Як перевірити, що method не викликався? Використати `mock.method.assert_not_called()`.

56. Що таке state testing? Перевірка видимого стану або результату системи після дії.

57. Що таке interaction testing? Перевірка взаємодій між об'єктами: хто, коли і з якими аргументами викликав залежність.

58. Коли interaction testing виправдане? На межах компонентів, коли правильний виклик є частиною контракту або треба перевірити відсутність небажаної дії.

59. Що таке patch? Тимчасова заміна об'єкта або імені на підміну на час тесту.

60. Чому patch потрібно виконувати там, де name використовується? Код звертається до імені у своєму namespace; заміна іншого namespace не змінить це посилання.

61. Що таке AsyncMock? Асинхронний test double, який поводиться як awaitable і відстежує await-виклики.

62. Як перевірити awaited call? Використати `async_mock.assert_awaited_once_with(...)`.

63. Що таке flaky test? Тест, результат якого нестабільно змінюється між запусками без змін коду.

64. Чому flaky tests небезпечні? Вони знижують довіру до CI, маскують справжні регресії та витрачають час на повторні запуски.

65. Що таке deterministic test? Тест із однаковим результатом для однакового коду й контрольованих вхідних даних.

66. Чому unit test не повинен залежати від Internet? Мережа додає повільність, нестабільність, зовнішній стан і недетерміновані помилки.

67. Коли в integration test доцільний real filesystem? Коли тестується реальний файловий контракт; використовується ізольований тимчасовий каталог.

68. Чому не потрібно mock-ати все? Надмірні mocks можуть тестувати лише з'єднання mocks і приховати помилки справжньої логіки.

69. Що таке over-mocking? Підміна такої кількості внутрішніх компонентів, що тест перестає перевіряти поведінку системи.

70. Що таке coverage? Метрика частки вимірюваного коду, який виконувався під час тестів.

71. Що таке statement coverage? Частка виконаних інструкцій коду.

72. Що таке branch coverage? Частка виконаних напрямків переходу, наприклад обох гілок умовного виразу.

73. Чим вони відрізняються? Statement coverage рахує рядки/інструкції, branch coverage окремо рахує альтернативні переходи умов.

74. Як отримати coverage через pytest-cov? Запустити `python -m pytest --cov=car_catalog --cov-branch --cov-report=term-missing --cov-report=html`.

75. Для чого --cov-report=term-missing? Показати coverage у терміналі та перелік рядків, які не були виконані.

76. Для чого HTML coverage report? Для інтерактивного перегляду покритих і пропущених рядків за файлами у браузері.

77. Чи гарантує 100% coverage відсутність bugs? Ні; він не оцінює якість assertions і не доводить правильність очікувань.

78. Чому meaningful assertions важливіші за coverage percentage? Тільки змістовна assertion порівнює результат із контрактом і може виявити неправильну поведінку.

79. Що таке coverage threshold? Мінімальний відсоток coverage, нижче якого звіт/CI завершується невдало.

80. Чому tests повинні бути незалежними один від одного? Тоді вони детерміновані, запускаються в будь-якому порядку й не приховують помилки через спільний стан.

11. Висновок

Створено й перевірено test suite для каталогу автомобілів із unit та integration tests, fixtures різних scopes, factory, parametrization, boundary/error checks, tmp_path, monkeypatch environment/config, Mock/MagicMock, side_effect, interaction assertions, configuration, файлового імпорту/експорту, branch і statement coverage. Повний suite проходить: 78 passed, 0 failed; coverage вище встановленого 80% threshold. Структурні прогалини branch coverage

описані окремо; AsyncMock обґрунтовано не застосовано, оскільки застосунок не має асинхронної поведінки.